# COVER_VALUE

## Overview

**cover_value** collects value coverage for the multi-bit data input. Users may define any number of values with the minimum & maximum number of hits for each of the predefined values. When ALL counters reach their minimum limits, the "covered" access event is asserted. When ONE of counters reaches its maximum limit, the "exceeded" access event is asserted. All coverage information can be accessed dynamically in order to tune test the generation process and define additional constraints and the length of simulation run.

## Syntax

```
cover_value #(bit_width,cover_num, ssize) InstName (clk, rst_n, data);
```

| **Parameters** | | |
|---|---|---|
| Bit_width | | Bit width of the data input |
| cover_num | | Maximum number of coverage items allowed. Default: 0 |
| Ssize | | String size for the name of a cover item. Default: 20 |
| **Interface** | | |
| clk | | Sampling clock for data |
| rst_n | | Active-low reset |
| data | | Input data signal, for coverage collection. Bit width of this signal has to be defined using a "bit_width" parameter. |
| **Access Tasks** | | |
| *Name* | *Arguments* | *Description* |
| reset | No | Resets coverage counters and the maximum & minimum limits |
| add | cover_name<br>cover_value<br>min_limit<br>max_limit | - Name of the coverage Item (with length <= ssize)<br>- Value to be covered<br>- Lower limit for the counter. Default: 0<br>- Upper limit for the counter. Default: 32'hFFFF_FFFF<br>-<br>Use an "add" access task to add functional Items to the cover_value coverage group.<br><br>Example: <InstName>.add(("VAL_10", 10, 20, 100);<br>– add coverage for value 10, with 20 minimum and 100 maximum hits |
| change | cover_name<br>cover_value<br>min_limit<br>max_limit | - name of the coverage Item (with length <= ssize)<br>- Value to be covered<br>- Lower limit for the counter. Default: 0<br>- Upper limit for the counter. Default: 32'hFFFF_FFFF<br>-<br>Use a "change" access task to modify the already existing functional Items within the cover_value coverage group.<br><br>Example: <InstName>.change(("VAL_10", 10, 30, 80);<br>– The change coverage for a value of 10 is to be 30 minimum and 80 maximum hits |
| report | No | Generates a coverage report in tabular format. See below an example of a report. |
| **Access Events** | | |
| covered | "Fires" when all vc_cover counters exceed their lower limits. Can be monitored externally to define the length of the test case execution based on the accumulated coverage information. | |

| | |
|---|---|
| exceeded | “Fires” when any of the counters exceed a predefined maximum limit. The maximum limits can be used as additional test-specific constraints. When it fires, “Exceeded_name” access register provides the name of the cover item exceeding the maximum limit. |
| **Access Variables** | |
| total_cov | Provided a constantly updated value of the total coverage (in percents). After a “covered” event, it has to be equal to 100 %. |
| exceeded_name | The name of the coverage item with the counter exceeding a predefined maximum value. |
| **Control Variables** | |
| Display_status | Set it to “1” to get more informational messages. Default: 0 |

## Verilog example

The example below shows how coverage information controls the simulation run. By monitoring a “covered” event, the test case is running until a defined coverage is accumulated. By monitoring an “exceeded” event, the test case exits upon the first violation.

```
module test;

reg clk; initial clk = 0;
reg [6:0] data;

parameter INIT = 0;
parameter S1   = 1;
parameter S2   = 2;
parameter S3   = 3;
parameter S4   = 4;
parameter S5   = 5;
parameter S6   = 6;
parameter S7   = 7;

always #10 clk <= ~clk;

always @(posedge clk)
  data <= ($random)%20;

//-------------------------------------
cover_value #(7,5) cov (clk, 1'b1, data);

initial begin
  cov.display_status = 0;
//         name       from  to  min max
//         ---------------------------
  cov.add("VAL 1",  1, 5,  100);
  cov.add("VAL 2",  2, 5,  100);
  cov.add("VAL 3",  3, 5,  100);
  cov.add("VAL 4",  4, 5,  100);
  cov.add("VAL 5",  5, 5,  100);
end
//-------------------------------------

initial forever begin
  #1000 cov.report;
end

// Exit on first maximum violation
always @(cov.exceeded) begin
```

```
  $display ("[%0t] TC INFO: MAXIMUM reached for '%s'",
      $time, cov.exceeded_name);
  $finish;
end

// Run till all functional coverage is collected; then exit
always @(cov.covered) begin
  $display("[%0t] TC INFO: Stimulus coverage reached, test may exit", $time);
  cov.report;
  $finish;
end

//If functional coverage is not collected, use timebomb
initial begin
  $dumpvars;
  #10000 $finish;
end

endmodule
```

## Value Coverage Report Example

```
                Value Coverage Report <test.cov.report>
------------------------------------------------------------------------
           Cover name       Count      Min_limit  Max_limit     Coverage
------------------------------------------------------------------------
               VAL 1         12          5        100          100 percent
               VAL 2          2          5        100           40 percent
               VAL 3          1          5        100           20 percent
               VAL 4          4          5        100           80 percent
               VAL 5          3          5        100           60 percent

Total Coverage:          60 percent
```